

PROYECTO 2 – DESARROLLO DE APLICACIÓN CON Java, MySQL y LLM

FASE 1 – ANÁLISIS DEL SISTEMA

1. Descripción del sistema

El sistema que se va a desarrollar es una aplicación de gestión para la empresa LevelUp Arcade, especializada en la venta de videojuegos y merchandising.

Actualmente la empresa gestiona su información mediante hojas de cálculo, lo que provoca errores, duplicidad de datos y pérdida de control del inventario y clientes.

El objetivo del sistema es crear una aplicación en Java conectada a una base de datos MySQL que permita gestionar de forma centralizada:

- Productos
- Clientes
- Proveedores

Además, incorporará funcionalidades de Inteligencia Artificial mediante OpenRouter para mejorar la gestión de productos.

2. Objetivos del sistema

- Digitalizar la gestión de inventario.
- Mejorar la gestión de clientes y proveedores.
- Reducir errores humanos.
- Automatizar tareas mediante IA.
- Centralizar toda la información en una base de datos.
- Facilitar el mantenimiento y escalabilidad del sistema.

3. Requisitos funcionales

Los requisitos funcionales describen qué hará el sistema.

RF1. Gestión de usuarios

El sistema permitirá registrar, modificar, eliminar y consultar usuarios.

RF2. Gestión de clientes

El sistema permitirá almacenar la información de los clientes, incluyendo nombre, teléfono y correo electrónico.

RF3. Gestión de productos

El sistema permitirá registrar videojuegos, consolas y accesorios con su precio, stock y categoría.

RF4. Gestión de proveedores

El sistema permitirá almacenar y administrar la información de los proveedores.

RF5. Gestión de pedidos

El sistema permitirá crear pedidos asociados a clientes y productos.

RF6. Control de stock

El sistema actualizará automáticamente el stock de productos después de cada venta o pedido.

RF7. Búsqueda de información

El sistema permitirá buscar productos, clientes y pedidos mediante filtros.

RF8. Conexión con base de datos

El sistema almacenará toda la información en una base de datos MySQL/MariaDB.

RF9. Inicio de sesión

El sistema permitirá iniciar sesión mediante usuario y contraseña.

RF10. Generación de informes

El sistema permitirá consultar listados de productos, clientes y pedidos.

4. Requisitos no funcionales

1) Requisitos de capacidad

- El sistema deberá soportar al menos 50 productos registrados.
- El tiempo de respuesta de las consultas no deberá superar los 3 segundos.
- La base de datos deberá permitir almacenar clientes, pedidos y productos sin pérdida de rendimiento.

2) Requisitos de interfaces

- El sistema se conectará a una base de datos MariaDB/MySQL mediante JDBC.
- El sistema será desarrollado en Java usando Eclipse.
- La interfaz utilizará formularios gráficos básicos.

3) Requisitos de operación

- El sistema deberá permitir iniciar y cerrar sesión.
- El usuario podrá utilizar teclado y ratón para navegar.
- El sistema deberá poder iniciarse desde cualquier ordenador con Java instalado.

4) Requisitos de recursos

- Se necesitará un servidor Ubuntu con MariaDB instalado.
- El sistema requerirá Java JDK y el conector JDBC de MySQL.
- Se necesitará conexión en red para acceder a la base de datos remota.

5) Requisitos de verificación

- El sistema deberá comprobar que los campos obligatorios estén completos.
- El sistema validará usuarios y contraseñas antes del acceso.

6) Requisitos de pruebas de aceptación

- El sistema será aceptado si permite gestionar correctamente productos, clientes y pedidos.
- Todas las operaciones CRUD deberán funcionar correctamente.

7) Requisitos de documentación

- El proyecto incluirá documentación técnica y manual de usuario.
- Se entregarán diagramas UML y scripts SQL.

8) Requisitos de seguridad

- El acceso al sistema requerirá autenticación mediante contraseña.
- Solo los usuarios autorizados podrán modificar datos.
- La base de datos estará protegida mediante usuarios y permisos.

9) Requisitos de transportabilidad

- El sistema podrá ejecutarse en Windows y Linux.
- La aplicación podrá trasladarse fácilmente usando Java.

10) Requisitos de calidad

- La interfaz deberá ser sencilla e intuitiva.
- El código seguirá una estructura MVC para facilitar la organización.

11) Requisitos de fiabilidad

- El sistema deberá mantener la información almacenada sin pérdidas.
- La aplicación deberá minimizar errores durante el uso normal.

12) Requisitos de mantenibilidad

- El código estará dividido en clases y paquetes organizados.
- La estructura permitirá añadir nuevas funciones fácilmente.

13) Requisitos de salvaguarda

- El sistema deberá evitar eliminar información accidentalmente.
- Se realizarán copias de seguridad de la base de datos.

5. Actores del sistema

Administrador

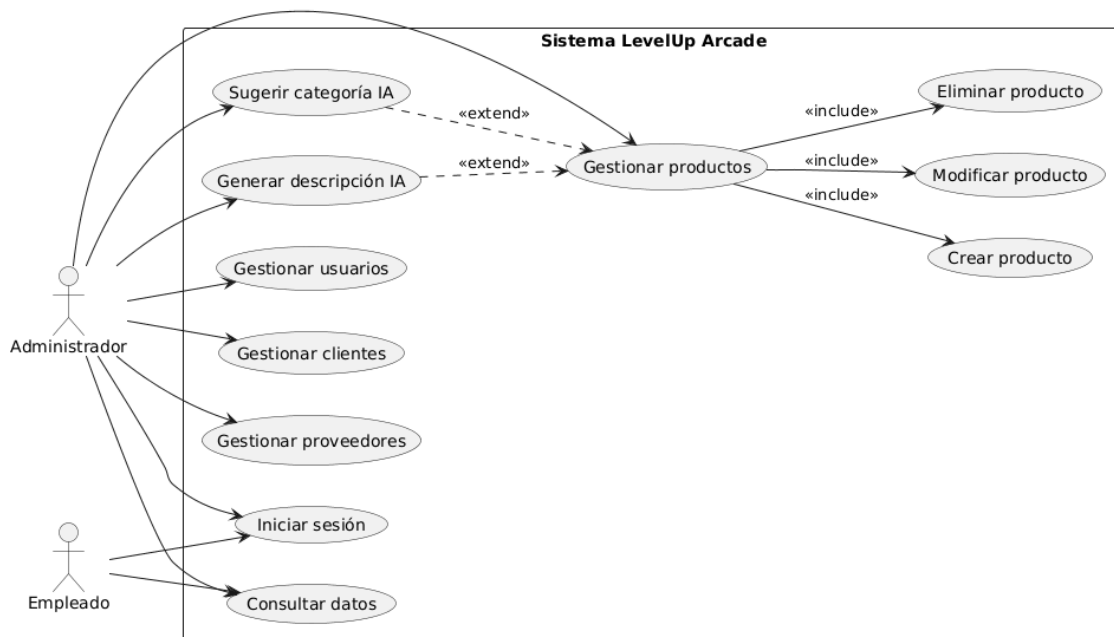
- Acceso total al sistema.
- Puede crear, modificar y eliminar cualquier dato.
- Gestiona usuarios.

Empleado

- Puede consultar datos.
- No puede modificar ni eliminar información.
- Acceso limitado a funcionalidades.

6. Casos de uso

Diagrama de casos de uso:



7. Historias de usuario

HU1 – Gestión de productos

Como administrador, quiero poder añadir, modificar y eliminar productos para mantener actualizado el inventario.

HU2 – Consulta de datos

Como empleado, quiero consultar productos, clientes y proveedores para atender a los clientes.

HU3 – Gestión de clientes

Como administrador, quiero gestionar clientes para mantener la base de datos actualizada.

HU4 – Gestión de proveedores

Como administrador, quiero gestionar proveedores para controlar el abastecimiento.

HU5 – IA para productos

Como administrador, quiero generar descripciones automáticas para ahorrar tiempo.

8. Backlog del proyecto (Jira):

Espacio / LevelUp Arcade

Tablero SRUM

ResumenCronogramaBacklogSprints activosCalendarioInformesMore9

Q Buscar en el backlog

VersiónEpicFiltros rápidos

FASE 1) ANÁLISIS DEL SISTEMA18 may – 19 may (5 actividades)

000Iniciar sprint

SRUM-1 funcionalesEN CURSO

SRUM-2 No funcionalesEN CURSO

SRUM-3 Identificar los actores ...EN CURSO

SRUM-4 diagrama de casos de ...EN CURSO

SRUM-5 Realizar una planificac...EN CURSO

+ Crear

5 actividades | Estimación: 0

FASE 2) DISEÑO DEL SISTEMA25 may – 26 may (4 actividades)

000

SRUM-7 Diseño de la base de ...TAREAS POR HACER

SRUM-8 Diagramas UML de las...TAREAS POR HACER

SRUM-9 Diagramas de flujoTAREAS POR HACER

SRUM-10 Mockup de la interfa...TAREAS POR HACER

+ Crear

4 actividades | Estimación: 0

FASE 3) INFRAESTRUCTURA Y ENTO20 may – 23 may (4 actividades)

000

SRUM-11 Instalar un servidor en Ubuntu ServerFINALIZADA23 may

SRUM-12 Conectarse remotamente al servidorEN CURSO23 may

SRUM-13 Crear la base de datos del sistemaEN CURSO23 may

SRUM-14 Ejecutar los scripts SQL creados en la Fase 2EN CURSO23 may

+ Crear

4 actividades | Estimación: 0

FASE 4) Codificación25 may – 27 may (6 actividades)

000

SRUM-15 Se implementará la aplicaciónTAREAS POR HA27 may

SRUM-16 El sistema deberá permitir el almacenamiento y r...TAREAS POR HA27 may

SRUM-17 Funcionalidad del sistemaTAREAS POR HA27 may

SRUM-18 Se deberán realizar validaciones y gestión de err...TAREAS POR HA27 may

SRUM-19 Se deberá documentar el código utilizando Java...TAREAS POR HA27 may

SRUM-20 El proyecto deberá utilizar control de versiones c...TAREAS POR HA27 may

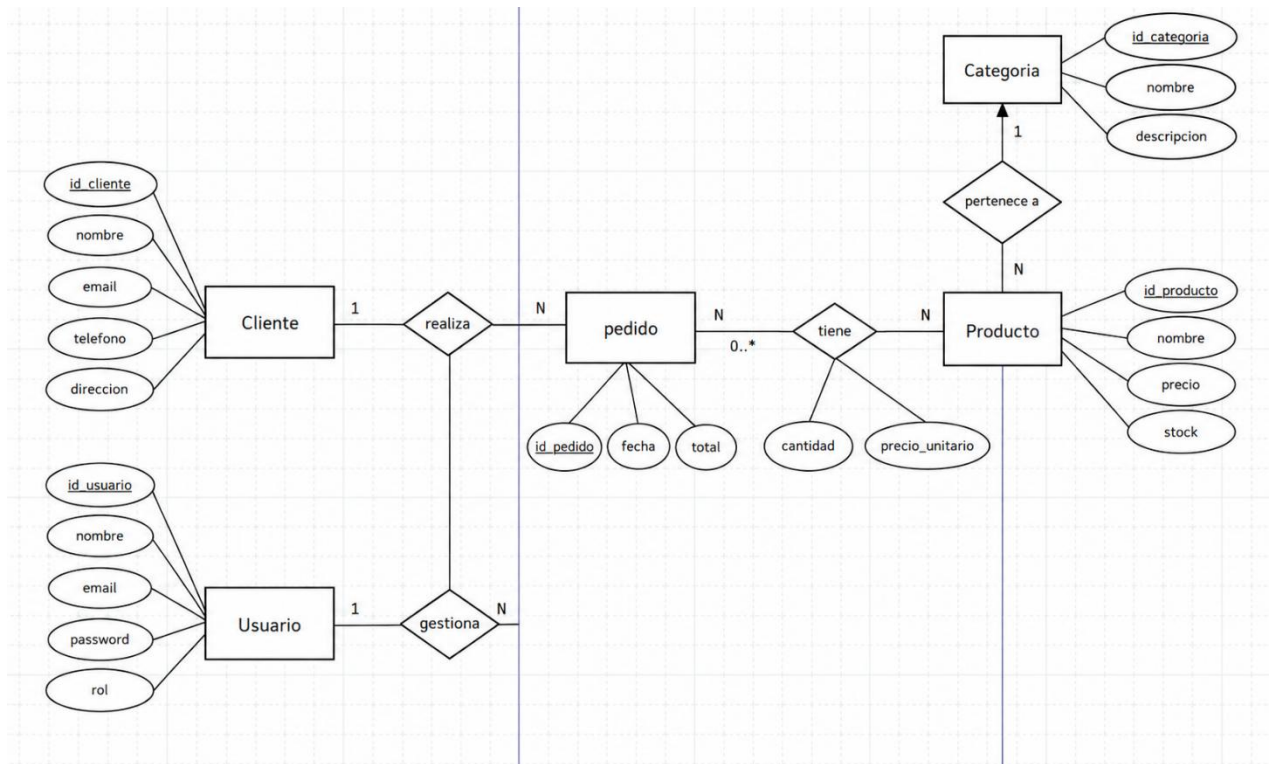
+ Crear

6 actividades | Estimación: 0

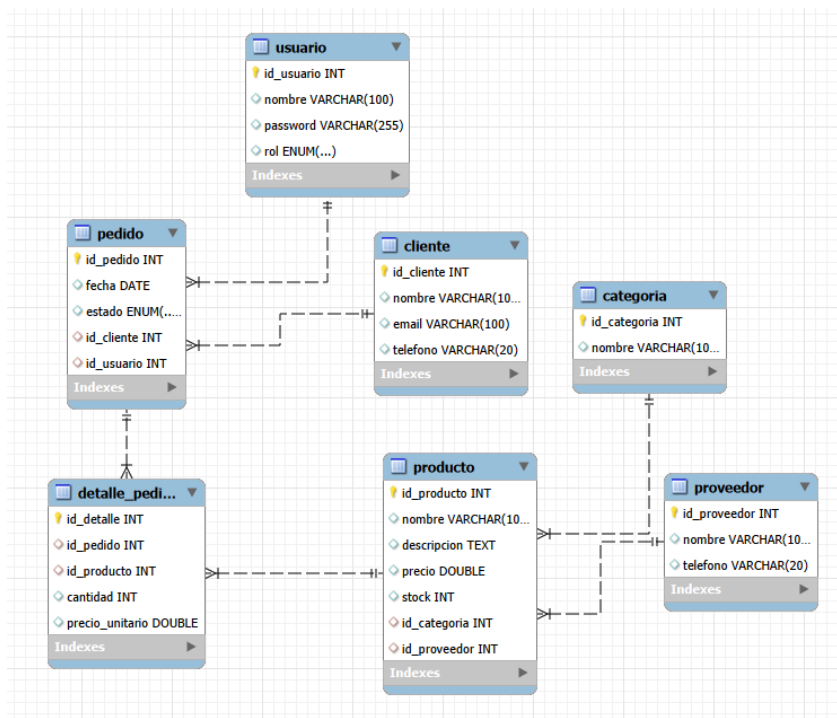
FASE 2) DISEÑO DEL SISTEMA

1. DISEÑO DE LA BASE DE DATOS

1.1 Modelo conceptual



1.2 Modelo lógico



1.3 Modelo físico:

```

1 • CREATE DATABASE IF NOT EXISTS levelup_arcade;
2 • USE levelup_arcade;
3 -- =====
4 -- USUARIOS (FASE 7 SEGURIDAD)
5 -- =====
6 • CREATE TABLE usuario (
7     id_usuario INT AUTO_INCREMENT PRIMARY KEY,
8     username VARCHAR(100) NOT NULL UNIQUE,
9     password VARCHAR(255) NOT NULL,
10    rol ENUM('ADMIN', 'EMPLEADO') NOT NULL
11 );
12 -- =====
13 -- CLIENTES
14 -- =====
15 • CREATE TABLE cliente (
16     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
17     nombre VARCHAR(100) NOT NULL,
18     email VARCHAR(100) NOT NULL UNIQUE,
19     telefono VARCHAR(20)
20 );
21 -- =====
22 -- PROVEEDORES
23 -- =====
24 • CREATE TABLE proveedor (
25     id_proveedor INT AUTO_INCREMENT PRIMARY KEY,
26     nombre VARCHAR(100) NOT NULL,
27     telefono VARCHAR(20)
28 );
29 -- =====
30 -- CATEGORÍAS
31 -- =====
32 • CREATE TABLE categoria (
33     id_categoria INT AUTO_INCREMENT PRIMARY KEY,
34     nombre VARCHAR(100) NOT NULL
35 );
36 -- =====
37 -- PRODUCTOS
38 -- =====
39 • CREATE TABLE producto (
40     id_producto INT AUTO_INCREMENT PRIMARY KEY,
41     nombre VARCHAR(100) NOT NULL,
42     descripcion TEXT,
43     precio DECIMAL(10,2) NOT NULL CHECK (precio >= 0),
44     stock INT NOT NULL CHECK (stock >= 0),
45     id_categoria INT NOT NULL,
46     id_proveedor INT NOT NULL,
47
48     FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)
49         ON DELETE CASCADE ON UPDATE CASCADE,
50
51     FOREIGN KEY (id_proveedor) REFERENCES proveedor(id_proveedor)
52         ON DELETE CASCADE ON UPDATE CASCADE
53 );
54 -- =====
55 -- PEDIDOS
56 -- =====
57 • CREATE TABLE pedido (
58     id_pedido INT AUTO_INCREMENT PRIMARY KEY,
59     fecha DATE NOT NULL,
60     estado ENUM('PENDIENTE', 'PREPARADO', 'ENVIADO', 'ENTREGADO', 'CANCELADO') NOT NULL,
61     id_cliente INT NOT NULL,
62     id_usuario INT NOT NULL,
63
64     FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
65         ON DELETE CASCADE ON UPDATE CASCADE,
66
67     FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
68         ON DELETE CASCADE ON UPDATE CASCADE
69 );
70 -- =====
71 -- DETALLE PEDIDO
72 -- =====
73 • CREATE TABLE detalle_pedido (
74     id_detalle INT AUTO_INCREMENT PRIMARY KEY,
75     id_pedido INT NOT NULL,
76     id_producto INT NOT NULL,
77     cantidad INT NOT NULL CHECK (cantidad > 0),
78     precio_unitario DECIMAL(10,2) NOT NULL CHECK (precio_unitario >= 0),
79
80     FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido)
81         ON DELETE CASCADE ON UPDATE CASCADE,
82
83     FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
84         ON DELETE CASCADE ON UPDATE CASCADE
85 );

```

1.4 Normalización (hasta 3FN)

1FN

- Todos los campos son atómicos (sin listas ni valores repetidos)

2FN

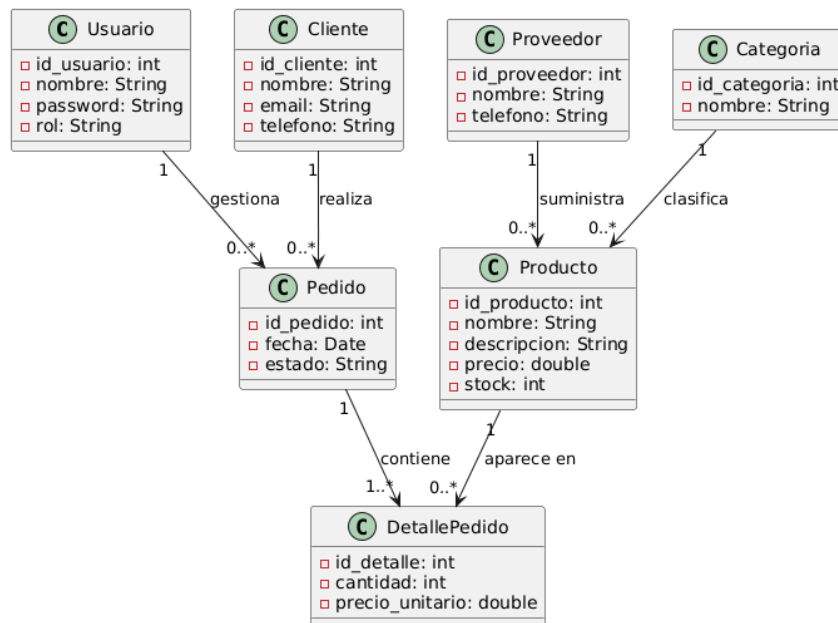
- Todas las tablas tienen clave primaria
- No hay dependencias parciales

3FN

- No existen dependencias transitivas:
 - categoría separada de producto
 - proveedor separado de producto
 - usuario independiente del resto

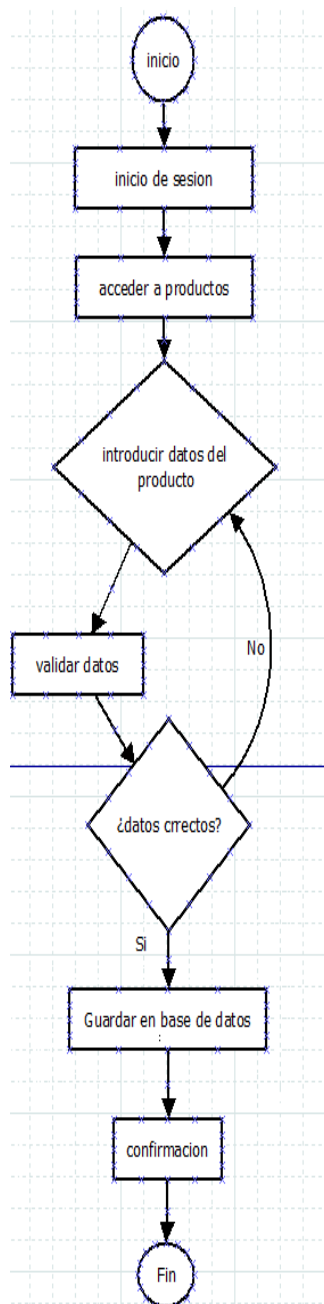
Resultado: base de datos en 3FN correcta

2. UML DE CLASES

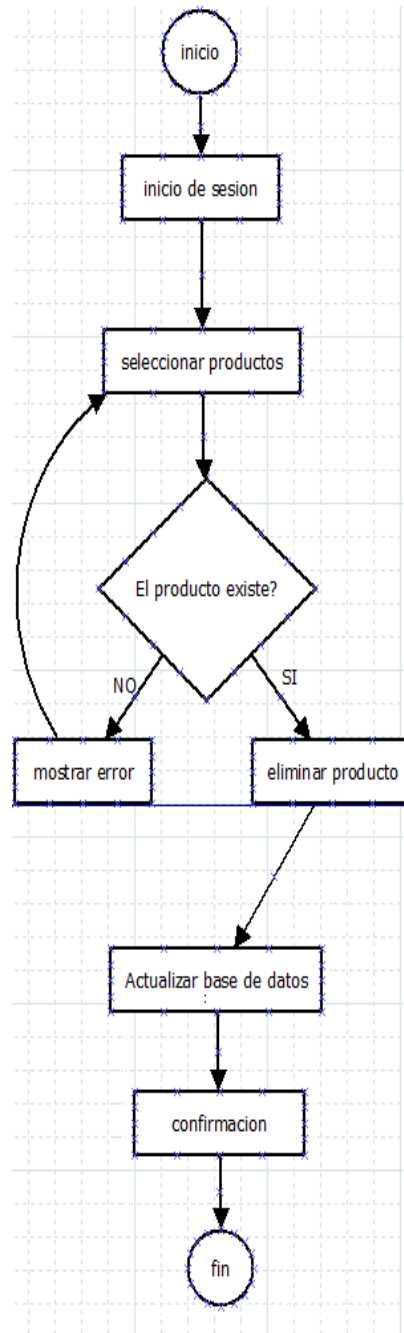


3. DIAGRAMAS DE FLUJO

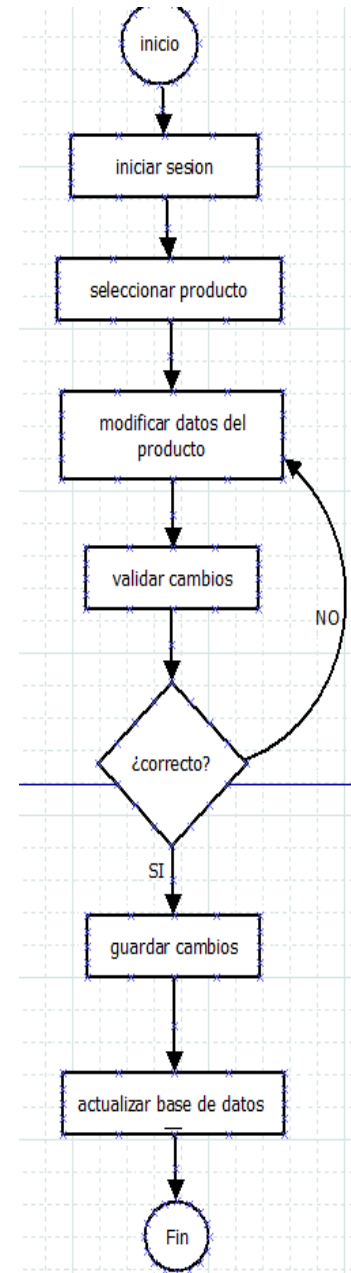
3.1 Alta de producto



3.2 Eliminar producto



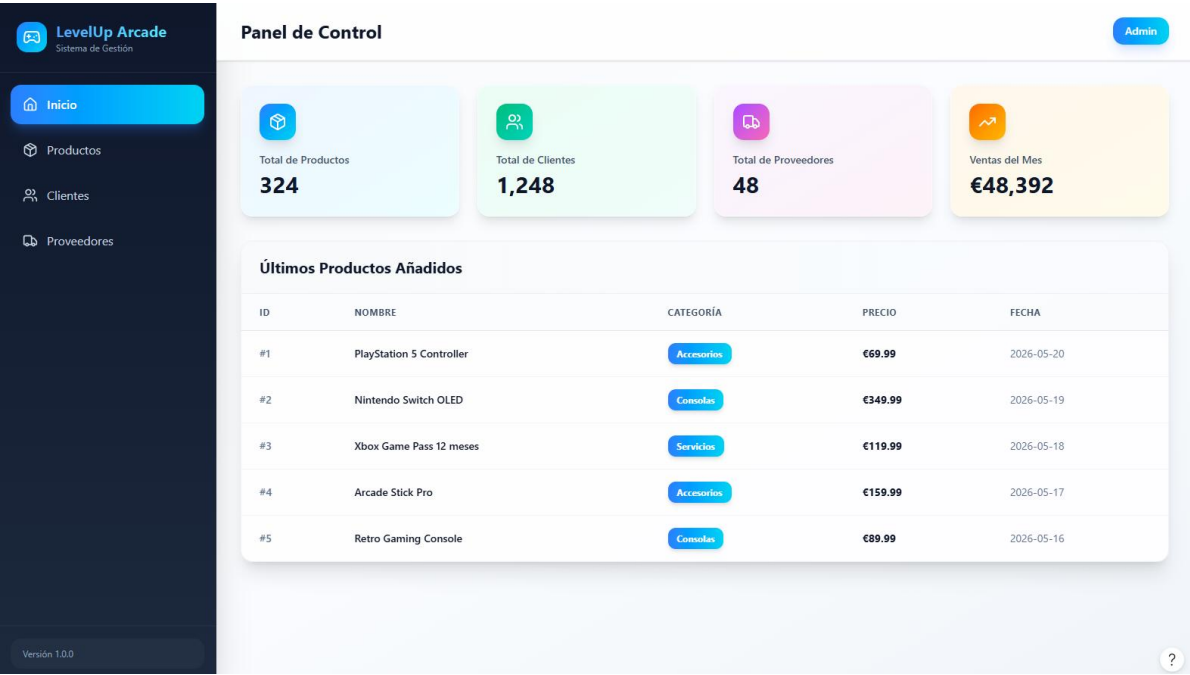
3.3 Modificar producto



Mockup de la interfaz gráfica:



Pantalla de inicio:



Pantallas de gestión (productos, clientes y proveedores):

LevelUp Arcade

Sistema de Gestión

Inicio

Productos

Cientes

Proveedores

Version 1.0.0

Panel de Control

Admin

Gestión de Productos

Administra tu inventario de videojuegos y accesorios

+ Añadir Producto

Buscar productos...

Todas las categorías

ID	NOMBRE	CATEGORÍA	PRECIO	STOCK	ESTADO	ACCIONES
#1	PlayStation 5 Controller	Accesorios	€69.99	45 (Alto)	Activo	 
#2	Nintendo Switch OLED	Consolas	€349.99	12 (Medio)	Activo	 
#3	Xbox Game Pass 12 meses	Servicios	€119.99	200 (Alto)	Activo	 
#4	Arcade Stick Pro	Accesorios	€159.99	8 (Bajo)	Activo	 
#5	Retro Gaming Console	Consolas	€89.99	3 (Bajo)	Inactivo	 
#6	Steam Deck 512GB	Consolas	€569.99	25 (Medio)	Activo	 
#7	Meta Quest 3	VR	€549.99	18 (Medio)	Activo	 
#8	Gaming Headset Pro	Accesorios	€129.99	67 (Alto)	Activo	 

Panel de Control

Admin

Gestión de Clientes

Administra tu base de clientes

+ Añadir Cliente

Buscar por nombre o email...

ID	NOMBRE COMPLETO	EMAIL	TELÉFONO	FECHA DE REGISTRO	ACCIONES
#1	Carlos Garcia	carlos.garcia@email.com	+34 612 345 678	2025-01-15	 
#2	Maria López	maria.lopez@email.com	+34 623 456 789	2025-02-20	 
#3	Juan Martinez	juan.martinez@email.com	+34 634 567 890	2025-03-10	 
#4	Ana Rodriguez	ana.rodriguez@email.com	+34 645 678 901	2025-04-05	 
#5	Pedro Sánchez	pedro.sanchez@email.com	+34 656 789 012	2025-05-01	 
#6	Laura Fernández	laura.fernandez@email.com	+34 667 890 123	2025-05-15	 

Panel de Control

Admin

Gestión de Proveedores

Administra tus proveedores y distribuidores

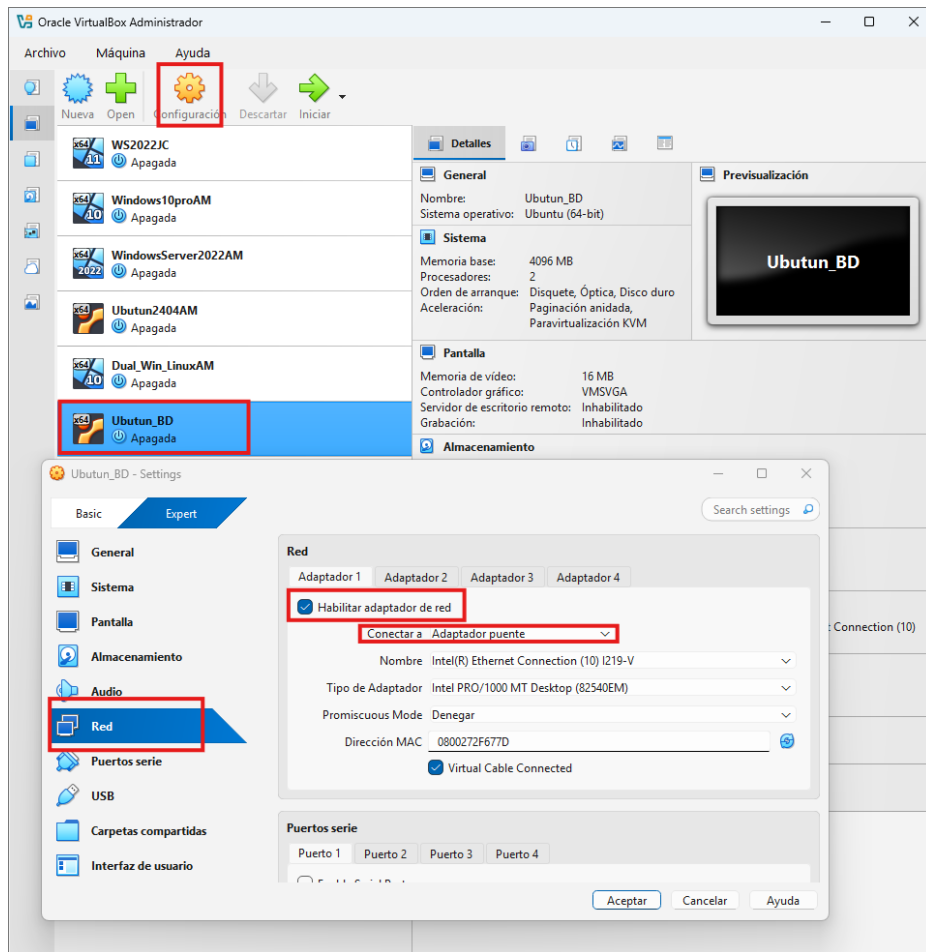
+ Añadir Proveedor

Buscar por nombre o empresa...

ID	NOMBRE DEL PROVEEDOR	EMPRESA	TELÉFONO	EMAIL	ACCIONES
#1	Roberto Garcia	TechGames Distribution	+34 910 123 456	roberto@techgames.com	 
#2	Elena Martin	Arcade Supplies Co.	+34 920 234 567	elena@arcadesupplies.com	 
#3	David López	Gaming World Import	+34 930 345 678	david@gamingworld.com	 
#4	Sara Rodriguez	Console Masters	+34 940 456 789	sara@consolemasters.com	 
#5	Miguel Sánchez	RetroGaming Wholesale	+34 950 567 890	miguel@retrogaming.com	 

FASE 3) INFRAESTRUCTURA Y ENTORNO

Configuración de su direccionamiento IP para que sea accesible:



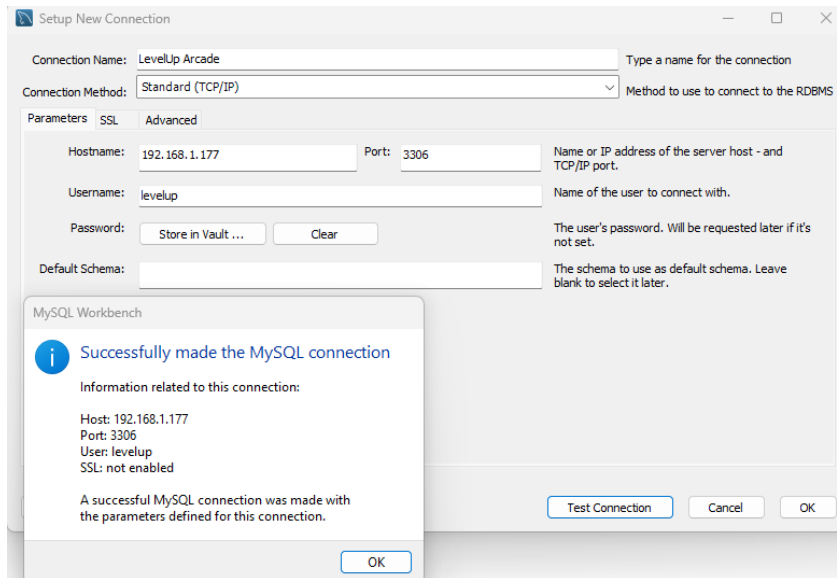
```
root@campusfp:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:2f:67:7d brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.177/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 254633sec preferred_lft 254633sec
    inet6 fe80::a00:27ff:fe2f:677d/64 scope link
        valid_lft forever preferred_lft forever
3: br-453c95896907: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 92:9f:c3:c0:95:9f brd ff:ff:ff:ff:ff:ff
    inet 172.19.0.1/16 brd 172.19.255.255 scope global br-453c95896907
        valid_lft forever preferred_lft forever
    inet6 fe80::909f:c3ff:fec0:959f/64 scope link
        valid_lft forever preferred_lft forever
4: br-a705dee639ad: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether ea:2a:0d:89:57:6b brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-a705dee639ad
        valid_lft forever preferred_lft forever
    inet6 fe80::e82a:dff:fe89:576b/64 scope link
        valid_lft forever preferred_lft forever
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 06:b6:89:b9:22:1f brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
6: veth51e66a3@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-453c95896907 state UP group default
    link/ether 12:19:64:3e:da:30 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::1019:64ff:fe3e:da30/64 scope link
        valid_lft forever preferred_lft forever
root@campusfp:~#
```

Instalación y configuración de MySQL:

En la maquina ha que ejecutar este comando;

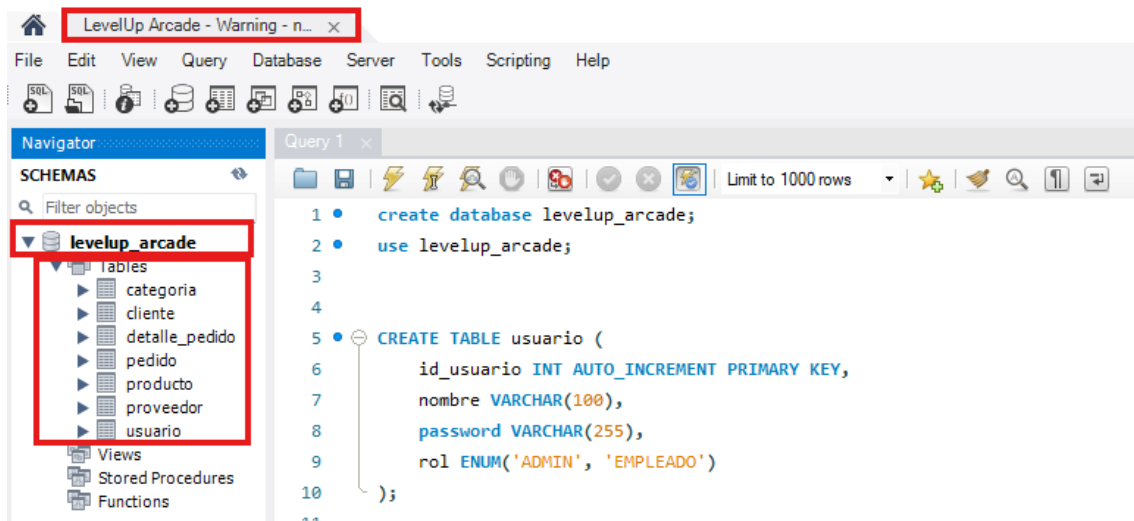
(sudo apt install mariadb-server).

Conectarse remotamente al servidor para:



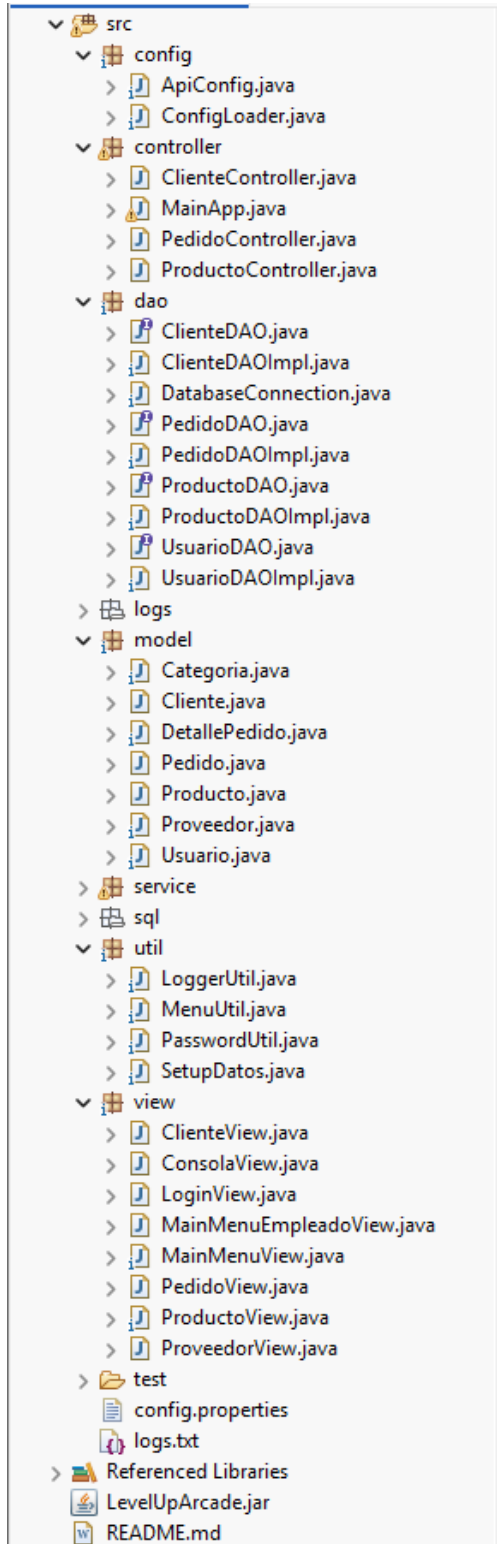
Crear la base de datos del sistema:

Ejecutar los scripts SQL creados en la Fase 2:



FASE 4) Codificación:

Se implementará la aplicación utilizando la arquitectura Modelo-Vista-Controlador (MVC), separando claramente:



El sistema deberá permitir el almacenamiento y recuperación de información mediante:

Base de datos MySQL:

MainApp [Java Application] C:\Program Files\Java\jdk-21.0.10\bin\javaw.exe (22 may 2026, 14:53:28 elapsed)

```

--- LEVELUP ARCADE ---
1. Clientes
2. Productos
3. Pedidos
0. Salir
Seleccione una opción: 1

--- NUEVO CLIENTE ---
Nombre: Ahmed
Email: ahmedss@gmail.com
Teléfono: 699239277
Cliente añadido correctamente.

--- LEVELUP ARCADE ---
1. Clientes
2. Productos
3. Pedidos
0. Salir
Seleccione una opción:

```

Ficheros de registro, donde se guardan logs de actividad y errores, permitiendo trazabilidad y auditoría del sistema:

```

*LoggerUtil.java X
1 package util;
2
3 import java.io.FileWriter;
4 import java.io.PrintWriter;
5 import java.time.LocalDateTime;
6
7 /**
8  * Clase para guardar logs del sistema.
9  */
10 public class LoggerUtil {
11
12     private static final String RUTA = "logs.txt";
13
14     public static void log(String mensaje) {
15
16         try (PrintWriter pw = new PrintWriter(new FileWriter(RUTA, true))) {
17
18             pw.println(
19                 "[" + LocalDateTime.now() + "] " + mensaje
20             );
21
22         } catch (Exception e) {
23             System.out.println("Error escribiendo log");
24         }
25     }
26 }

```

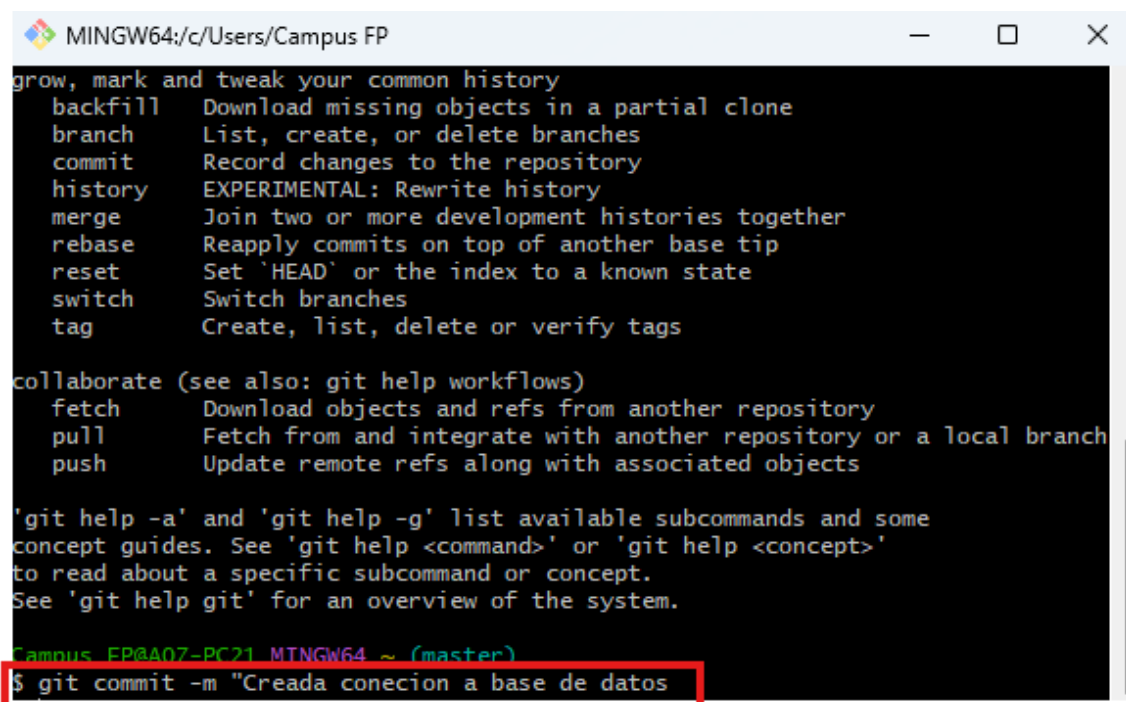
Se deberá documentar el código utilizando JavaDoc:

```

1 package dao;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import config.ConfigLoader;
6
7 /**
8  * Conexión única a MySQL (Singleton simple)
9  */
10 public class DatabaseConnection {
11
12     private static Connection conn;
13
14     public static Connection getConnection() {
15
16         try {
17             if (conn == null || conn.isClosed()) {
18                 conn = DriverManager.getConnection(
19                     ConfigLoader.get("db.url"),
20                     ConfigLoader.get("db.user"),
21                     ConfigLoader.get("db.password")
22                 );
23             }
24         } catch (Exception e) {
25             System.out.println("Error conexión DB: " + e.getMessage());
26         }
27
28         return conn;
29     }
30 }

```

El proyecto deberá utilizar control de versiones con Git, realizando commits frecuentes y organizados:



```

MINGW64:/c/Users/Campus FP
grow, mark and tweak your common history
  backfill  Download missing objects in a partial clone
  branch    List, create, or delete branches
  commit    Record changes to the repository
  history    EXPERIMENTAL: Rewrite history
  merge     Join two or more development histories together
  rebase    Reapply commits on top of another base tip
  reset     Set 'HEAD' or the index to a known state
  switch    Switch branches
  tag       Create, list, delete or verify tags

collaborate (see also: git help workflows)
  fetch     Download objects and refs from another repository
  pull      Fetch from and integrate with another repository or a local branch
  push      Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.

Campus FP@A07-PC21 MINGW64 ~ (master)
$ git commit -m "Creada conexion a base de datos"

```

FASE 5) INTELIGENCIA ARTIFICIAL

1. Regístrate en OpenRouter.ai y obtén tu API Key personal:

The screenshot shows the OpenRouter.ai interface for managing API keys. The browser address bar is highlighted with a red box, showing the URL `openrouter.ai/workspaces/default/keys`. The page title is "API Keys" and the subtitle is "Create and manage your API keys." A red box highlights the "+ New Key" button in the top right corner. A modal form is open for creating a new key. The form fields are: Name (filled with "LevelUp Arcade"), Credit limit (optional) (filled with "Leave blank for unlimited"), Reset limit every... (filled with "N/A"), and Expiration (filled with "No expiration"). A red box highlights the "Create" button at the bottom right of the modal.

2. Crea una clase LlmService.java. Implementa en ella un método que tome un prompt:



```

1 package service;
2
3 import config.ApiConfig;
4
5 import java.io.BufferedReader;
6 import java.io.InputStream;
7 import java.io.InputStreamReader;
8 import java.io.OutputStream;
9 import java.net.HttpURLConnection;
10 import java.net.URL;
11
12 public class LlmService {
13
14     private static final String API_URL =
15         "https://openrouter.ai/api/v1/chat/completions";
16
17     public String enviarPrompt(String prompt) {
18         try {
19             // =====
20             // CONEXIÓN HTTP
21             // =====
22             URL url = new URL(API_URL);
23
24             HttpURLConnection conn =
25                 (HttpURLConnection) url.openConnection();
26
27             conn.setRequestMethod("POST");
28             conn.setRequestProperty("Authorization",
29                 "Bearer " + ApiConfig.API_KEY);
30
31             conn.setRequestProperty("Content-Type",
32                 "application/json");
33
34             conn.setDoOutput(true);
35
36             // =====
37             // JSON REQUEST
38             // =====
39             String body = ""
40                 {
41                     "model": "openai/gpt-3.5-turbo",
42                     "messages": [
43                         {
44                             "role": "user",
45                             "content": "%s"
46                         }
47                     ]
48                 }
49             }
50             ""}.formatted(prompt);
51
52             OutputStream os = conn.getOutputStream();
53             os.write(body.getBytes("utf-8"));
54             os.close();
55
56
57             // =====
58             // RESPUESTA HTTP
59             // =====
60             int code = conn.getResponseCode();
61
62             InputStream is;
63
64             if (code >= 200 && code < 300) {
65                 is = conn.getInputStream();
66             } else {
67                 is = conn.getErrorStream();
68             }
69
70             BufferedReader br = new BufferedReader(
71                 new InputStreamReader(is, "utf-8")
72             );
73
74             StringBuilder response = new StringBuilder();
75             String line;
76
77             while ((line = br.readLine()) != null) {
78                 response.append(line);
79             }
80
81             br.close();
82
83             // =====
84             // EXTRAER TEXTO IA
85             // =====
86             return extraerContenido(response.toString());
87
88         } catch (Exception e) {
89             e.printStackTrace();
90             return "Error IA: " + e.getMessage();
91         }
92     }

```

```

93
94 // =====
95 // PARSEO SIMPLE JSON
96 // =====
97 private String extraerContenido(String json) {
98
99     try {
100         String key = "\"content\": \"";
101
102         int start = json.indexOf(key);
103
104         if (start == -1) {
105             return "No se pudo leer respuesta IA";
106         }
107
108         start += key.length();
109
110         int end = json.indexOf("\"", start);
111
112         if (end == -1) {
113             return "No se pudo leer respuesta IA";
114         }
115
116         return json.substring(start, end);
117     } catch (Exception e) {
118         return "Error parseando respuesta IA";
119     }
120 }
121
122 }

```

3. Generar Descripción de Producto con IA:

4. Sugerir Categoría para Producto con IA:

```

154 // =====
155 // IA (FASE 5)
156 // =====
157 private void generarDescripcionIA() {
158
159     if (txtNombre.getText().isEmpty()) {
160         JOptionPane.showMessageDialog(this,
161             "Introduce un nombre de producto");
162         return;
163     }
164
165     LlmService ia = new LlmService();
166
167     String prompt =
168         "Genera una descripción corta y profesional del siguiente producto: "
169         + txtNombre.getText();
170
171     String respuesta = ia.enviarPrompt(prompt);
172
173     JOptionPane.showMessageDialog(this, respuesta);
174 }
175
176 private void sugerirCategoriaIA() {
177
178     if (txtNombre.getText().isEmpty()) {
179         JOptionPane.showMessageDialog(this,
180             "Introduce un nombre de producto");
181         return;
182     }
183
184     LlmService ia = new LlmService();
185
186     String prompt =
187         "Dime la categoría de tienda adecuada para este producto: "
188         + txtNombre.getText();
189
190     String respuesta = ia.enviarPrompt(prompt);
191
192     JOptionPane.showMessageDialog(this, respuesta);
193 }
194 }

```

FASE 6) PRUEBAS Y CALIDAD

- Definir los casos de pruebas. Deben incluir: operaciones CRUD, validación de datos y casos de error y situaciones límite

1. CRUD Productos

ID	CASO DE PRUEBA	ENTRADA	RESULTADO ESPERADO	TIPO
CP01	Insertar producto válido	Nombre, precio > 0, stock > 0, categoría existente	Producto insertado correctamente	CREATE
CP02	Listar productos	Sin parámetros	Lista de productos mostrada	READ
CP03	Buscar producto por ID válido	ID existente	Producto devuelto correctamente	READ
CP04	Actualizar producto válido	Datos correctos + ID existente	Producto actualizado	UPDATE
CP05	Eliminar producto válido	ID existente	Producto eliminado	DELETE
CP06	Insertar producto sin nombre	Nombre vacío	Error de validación	ERROR
CP07	Insertar producto con precio negativo	Precio < 0	Error de validación	ERROR
CP08	Insertar producto con stock negativo	Stock < 0	Error de validación	ERROR

2. CRUD Clientes

ID	CASO DE PRUEBA	ENTRADA	RESULTADO ESPERADO	TIPO
CP09	Insertar cliente válido	Datos correctos	Cliente insertado	CREATE
CP10	Listar clientes	Sin parámetros	Lista mostrada	READ
CP11	Buscar cliente por ID	ID existente	Cliente encontrado	READ
CP12	Actualizar cliente	Datos válidos	Cliente actualizado	UPDATE
CP13	Eliminar cliente	ID existente	Cliente eliminado	DELETE
CP14	Email inválido	Email sin formato correcto	Error de validación	ERROR
CP15	Nombre vacío	""	Error de validación	ERROR

3. CRUD Proveedores

ID	CASO DE PRUEBA	ENTRADA	RESULTADO ESPERADO	TIPO
CP16	Insertar proveedor válido	Datos correctos	Proveedor insertado	CREATE
CP17	Listar proveedores	Sin parámetros	Lista mostrada	READ
CP18	Eliminar proveedor inexistente	ID no existente	Mensaje error controlado	ERROR

4. Validaciones de datos

ID	CASO DE PRUEBA	ENTRADA	RESULTADO ESPERADO	TIPO
CP19	Producto nulo	null	Error controlado	ERROR
CP20	Nombre vacío	""	Error de validación	ERROR
CP21	Precio negativo	-10	Error de validación	ERROR
CP22	Stock negativo	-5	Error de validación	ERROR
CP23	ID inválido	0 o negativo	Error de validación	ERROR

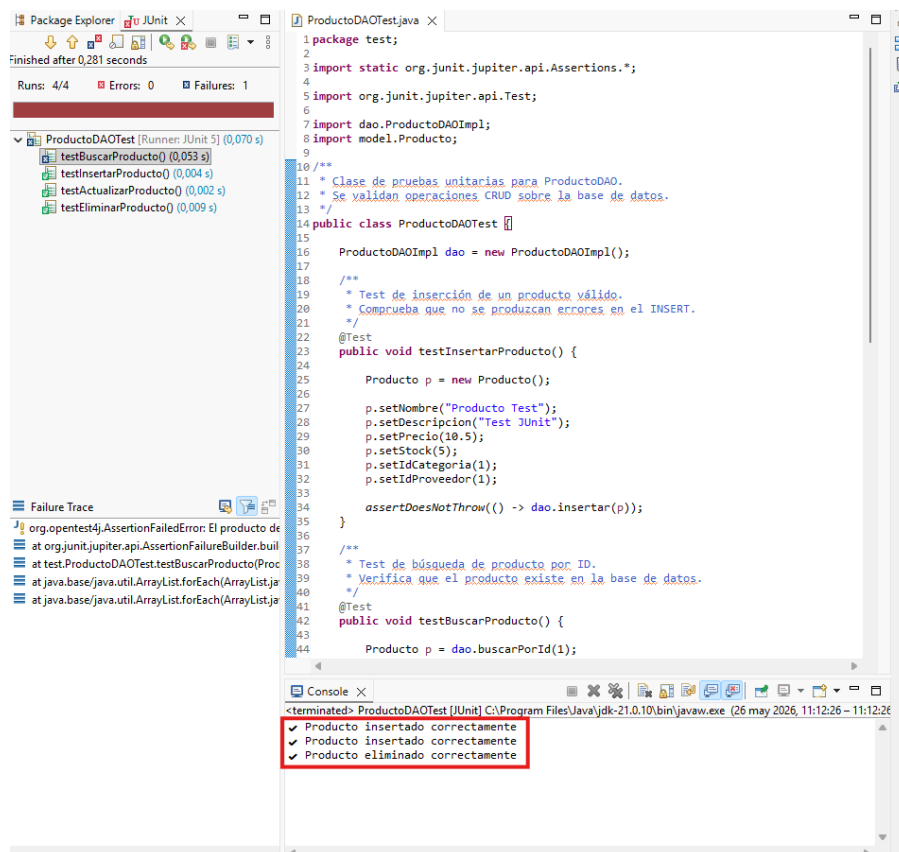
5. Casos de error (excepciones)

ID	CASO DE PRUEBA	ENTRADA	RESULTADO ESPERADO	TIPO
CP24	Base de datos caída	Sin conexión MySQL	Error controlado sin crash	ERROR
CP25	Foreign Key inválida	id_categoria inexistente	Error SQL controlado	ERROR
CP26	Query incorrecta	SQL mal formado	Excepción capturada	ERROR
CP27	NullPointerException	Objeto null	Error controlado	ERROR

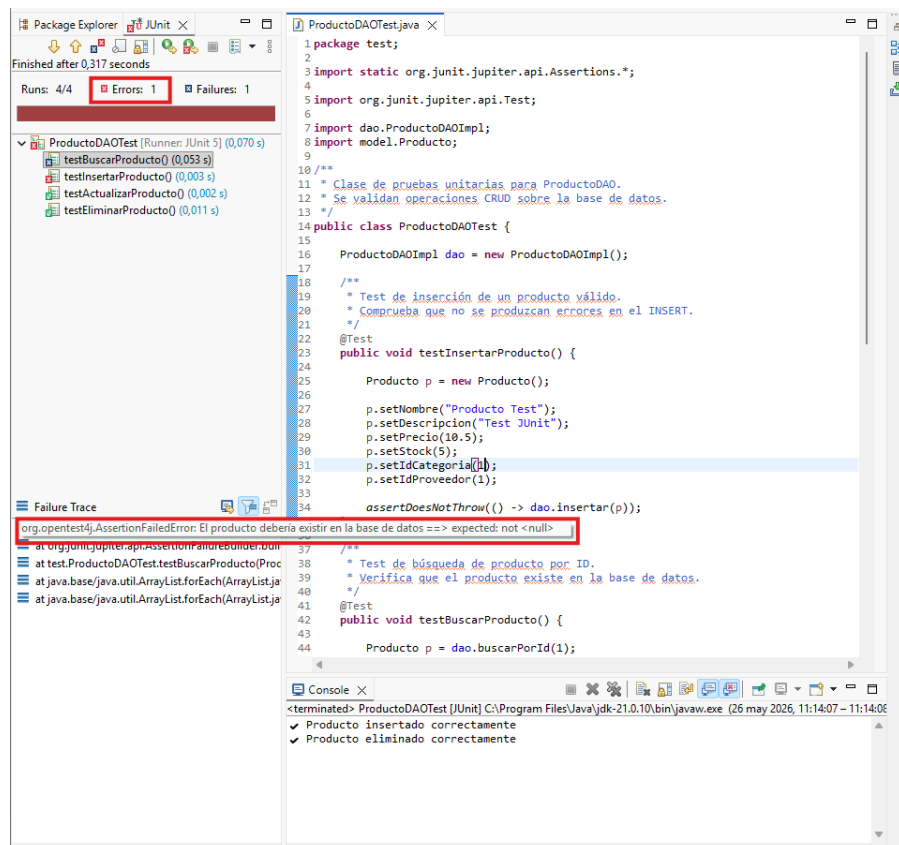
6. Casos límite (Fase 6)

ID	CASO DE PRUEBA	ENTRADA	RESULTADO ESPERADO	TIPO
CP28	Nombre muy largo	> 255 caracteres	Error o truncamiento controlado	LÍMITE
CP29	Stock = 0	0	Producto permitido o controlado	LÍMITE
CP30	Precio = 0	0	Permitido o validado según reglas	LÍMITE
CP31	Mucha carga de datos	1000 inserts	Sistema estable	LÍMITE
CP32	Listar BD grande	Muchos registros	Rendimiento aceptable	LÍMITE

- Implementar pruebas unitarias utilizando JUnit.



- Realizar depuración para corregir los errores detectados



- En el servidor Ubuntu se implementará un sistema de copias de seguridad automáticas de la base de datos. Para ello se creará un script de backup y Se programará una tarea automática (cron) para su ejecución periódica.

1. CREAR CARPETA DE BACKUPS En Ubuntu Server:

```
root@campusfp:~# sudo mkdir -p /home/levelup/backups
root@campusfp:~# sudo chmod 777 /home/levelup/backups
```

2. Crear archivo backup_levelup.sh:

```
GNU nano 7.2 backup_levelup.sh
#!/bin/bash

# BACKUP LEVELUP ARCADE

DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_DIR="/home/levelup/backup"
DB_NAME="levelup_arcade"
DB_USER="levelup"
DB_PASS="1234"

#Crear directorio si no existe
mkdir -p $BACKUP_DIR

#Crear backup
mysqldump -u $DB_USER -p$DB_PASS $DB_NAME > $BACKUP_DIR/backup_$DATE.sql

# Mensaje de confirmacion
echo "Backup realizado correctamente: backup_$DATE.sql"
```

3. DAR PERMISOS AL SCRIPT:

```
root@campusfp:~# chmod +x backup_levelup.sh
```

4. PROGRAMAR BACKUP AUTOMÁTICO

```
GNU nano 7.2 /tmp/crontab.Aaz11A/crontab *
0 2 * * * /home/levelup/backup_levelup.sh

root@campusfp:~# ./backup_levelup.sh
Backup realizado correctamente: backup_2026-05-26_11-11-53.sql
```


FASE 7) SEGURIDAD

- Crear un sistema de login de usuarios, en el que habrá dos roles bien diferenciados:

1. Administrador: podrá realizar todas las operaciones del sistema (CRUD completo y gestión de datos).

2. Empleado: solo podrá realizar consultas de datos (lectura), sin permisos de modificación:

```
MainApp.java X
19 // =====
20 // LOGIN (FASE 7)
21 // =====
22 LoginView loginView = new LoginView();
23 loginView.setVisible(true);
24
25 // Espera simple
26 while (loginView.isVisible()) {
27     try {
28         Thread.sleep(100);
29     } catch (Exception e) {
30         e.printStackTrace();
31     }
32 }
33
34 // =====
35 // CONTROL DESPUÉS DEL LOGIN
36 // =====
37
38 Usuario usuario = loginView.getUsuarioLogueado();
39
40 if (usuario == null) {
41     System.out.println("Acceso denegado");
42     System.exit(0);
43 }
44
45 System.out.println("Bienvenido " + usuario.getUsername() + " - " + usuario.getRc
46
47 Scanner sc = new Scanner(System.in);
48
49 ConsolaView view = new ConsolaView();
50
51 ClienteController clienteController = new ClienteController();
52 ProductoController productoController = new ProductoController();
53 PedidoController pedidoController = new PedidoController();
54
55 int opcion;
56
57 do {
58
59     view.menu();
60
61     System.out.print("Seleccione una opción: ");
62     opcion = sc.nextInt();
63     sc.nextLine();
64
65     switch (opcion) {
66
```

- Las contraseñas de los usuarios deberán almacenarse de forma segura. No podrán guardarse en texto plano, sino mediante técnicas de cifrado o hash.

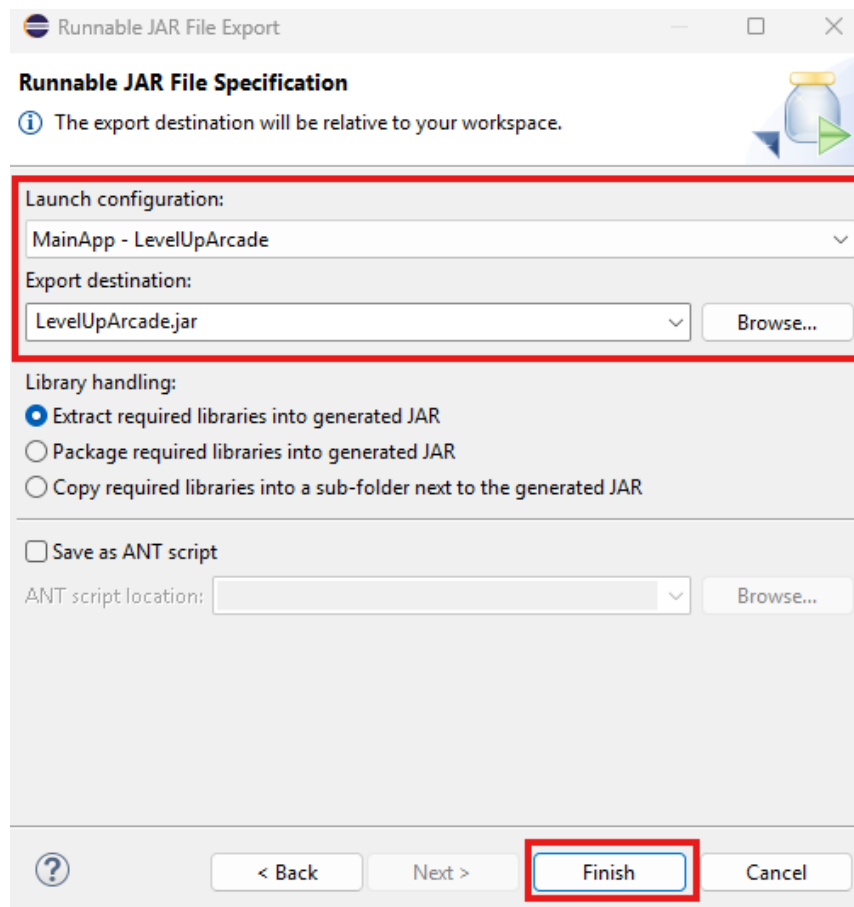
```

1  • CREATE DATABASE IF NOT EXISTS levelup_arcade;
2  USE levelup_arcade;
3
4  -- =====
5  -- USUARIOS (FASE 7 SEGURIDAD)
6  -- =====
7  • CREATE TABLE usuario (
8      id_usuario INT AUTO_INCREMENT PRIMARY KEY,
9      username VARCHAR(100) NOT NULL UNIQUE,
10     password VARCHAR(255) NOT NULL,
11     rol ENUM('ADMIN', 'EMPLEADO') NOT NULL
12 );
13
85 // =====
86 // LOGIN (FASE 7)
87 // =====
88
89 /**
90  * Valida login de usuario.
91  *
92  * @param username nombre de usuario
93  * @param password contraseña en texto plano
94  * @return Usuario si es válido, null si no existe
95  */
96 public Usuario login(String username, String password) {
97
98     String sql = "SELECT * FROM usuario WHERE username=? AND password=?";
99
100    try (PreparedStatement ps = conn.prepareStatement(sql)) {
101
102        ps.setString(1, username);
103        ps.setString(2, PasswordUtil.hashPassword(password));
104
105        ResultSet rs = ps.executeQuery();
106
107        if (rs.next()) {
108
109            Usuario u = new Usuario();
110
111            u.setIdUsuario(rs.getInt("id_usuario"));
112            u.setUsername(rs.getString("username"));
113            u.setPassword(rs.getString("password"));
114            u.setRol(rs.getString("rol"));
115
116            return u;
117        }
118    } catch (Exception e) {
119        System.out.println("Error en login: " + e.getMessage());
120    }
121
122    return null;
123 }
124 }
125 }

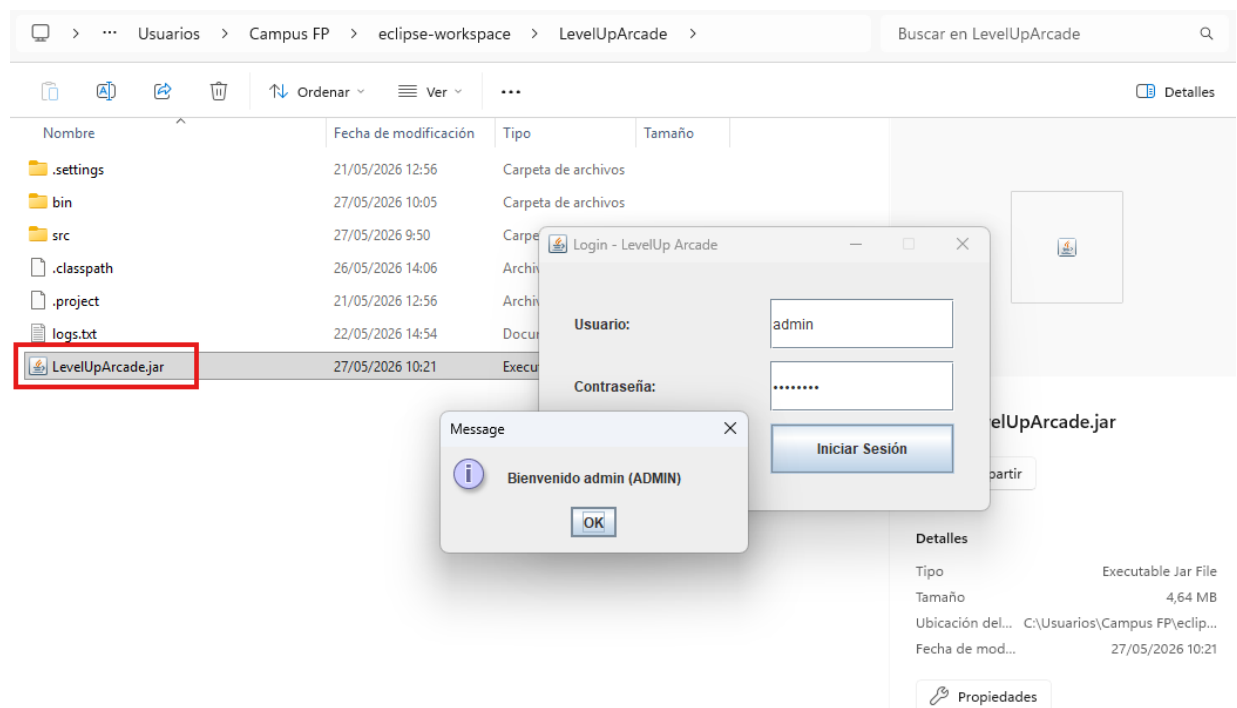
```

FASE 8) DESPLIEGUE

- Generar el archivo ejecutable del sistema en formato .jar.



- Verificar su correcto funcionamiento en un entorno externo al desarrollo.



- **Elaborar un manual de usuario que incluya:**

- o **Descripción del sistema**

- o **Instrucciones de instalación y ejecución**

- o **Explicación de las funcionalidades principales**

- o **Uso de las diferentes opciones del sistema:**

LevelUp Arcade es una aplicación de escritorio desarrollada en Java que permite la gestión de una tienda tipo arcade. El sistema incluye un inicio de sesión de usuarios con distintos roles (ADMIN y EMPLEADO), y a partir de ahí se puede acceder a las distintas funciones del programa. Entre ellas se encuentran la gestión de productos, clientes y pedidos, todo ello mediante una interfaz gráfica sencilla. Además, el sistema incorpora una integración con IA (OpenRouter) que permite generar automáticamente descripciones de productos.

Para instalar y ejecutar la aplicación es necesario disponer de Java instalado, una base de datos MySQL o MariaDB con la base de datos importada llamada “levelup_arcade”, el archivo ejecutable LevelUpArcade.jar y el archivo config.properties en la misma carpeta. Este archivo de configuración contiene los datos de conexión a la base de datos y la configuración de la IA. Una vez preparado todo, la aplicación se ejecuta desde consola con el comando `java -jar LevelUpArcade.jar`.

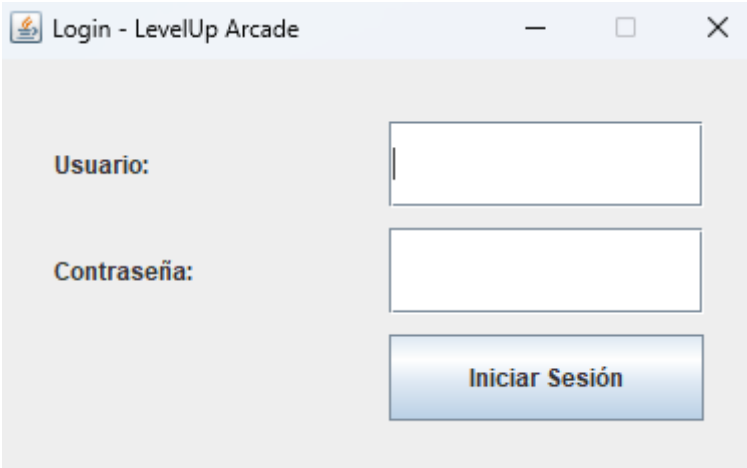
El sistema permite realizar las principales operaciones de gestión. En productos se pueden crear, editar, eliminar y listar artículos. En clientes se permite su alta, modificación, eliminación y consulta. En pedidos se pueden crear pedidos seleccionando cliente y productos, calculando automáticamente el total. Todo el sistema funciona de forma centralizada desde el menú principal.

El uso del sistema es sencillo: al iniciar la aplicación aparece una pantalla de login donde el usuario introduce sus credenciales. Dependiendo del rol asignado, tendrá acceso completo (ADMIN) o limitado (EMPLEADO).

Una vez dentro, el usuario puede navegar por las distintas opciones del sistema para realizar las operaciones necesarias de forma rápida y organizada.

La fase 9 la vas a hacer tu

FASE 10) INTERFAZ GRÁFICA (OPCIONAL)



A screenshot of a login window titled "Login - LevelUp Arcade". It features two input fields: "Usuario:" and "Contraseña:". Below the password field is a blue button labeled "Iniciar Sesión".



A screenshot of the main menu window titled "LevelUp Arcade". It displays five large, blue, rectangular buttons stacked vertically, each containing a menu item: "Productos", "Clientes", "Pedidos", "Proveedores", and "Salir".

Gestión de Productos

GESTIÓN DE PRODUCTOS

ID	Nombre	Precio	Stock
----	--------	--------	-------

Nombre:

Precio:

Stock:

Agregar

Eliminar

Limpiar

IA Descripción

IA Categoría

Cientes

GESTIÓN DE CLIENTES

ID	Nombre	Email
----	--------	-------

Nombre:

Email:

Agregar

Eliminar

Pedidos

GESTIÓN DE PEDIDOS

ID	Cliente	Fecha	Total
----	---------	-------	-------

Nuevo Pedido

Proveedores

GESTIÓN DE PROVEEDORES

ID	Nombre	Empresa
----	--------	---------

Nombre:

Empresa:

Agregar

Eliminar